

NK-7 — Guía de Sebastian para la Exposición

Léela completa una vez. Después solo necesitas repasar los recuadros marcados con .

Primero lo primero: ¿Qué es NK-7?

Es un **videojuego 2D** hecho en **Godot** (un programa para hacer videojuegos, como Unity pero gratis).

El personaje se llama **Kai**, es un técnico atrapado en una instalación industrial. Hay robots enemigos, láseres peligrosos y puertas bloqueadas. El jugador tiene que avanzar por los niveles esquivando o eliminando todo eso.

Eso es todo lo que necesitas saber del juego para explicar tu parte.

¿Qué te toca explicar a ti?

- Tienes **dos temas**:
- 1. **La IA de los enemigos** — cómo piensan los robots
 - 2. **El sistema de puertas** — cómo funcionan las puertas del juego

Tiempo: **~3 minutos**. No es mucho. Abajo te explico cada cosa con palabras normales.

TEMA 1: La IA de los enemigos (los robots Ukibuki)

¿Qué es "IA" en un videojuego?

IA = Inteligencia Artificial. En videojuegos no es nada mágico — es simplemente **una lista de reglas** que le dice al enemigo qué hacer según la situación.

Ejemplo de la vida real:

"Si hay alguien en mi cuarto → grito. Si no hay nadie → duermo."

Los robots del juego hacen lo mismo, pero con más estados.

Los estados del robot (la "lista de reglas")

El robot tiene **7 estados**. Piénsalo como el humor de una persona:

Estado	En español normal	Qué hace
IDLE	Quieto	Está parado, mirando si aparece Kai
PATROL	Patrullando	Camina de un lado al otro como guardia

Estado	En español normal	Qué hace
ALERT	Alerta	Vio algo sospechoso, se prepara
CHASE	Persiguiendo	Detectó a Kai, lo persigue corriendo
ATTACK	Atacando	Está cerca, dispara
DAMAGED	Golpeado	Recibió daño, se detiene un momento
DESTROYED	Muerto	Animación de explosión, desaparece

El robot **cambia de estado automáticamente** según lo que pasa. Esto se llama **Máquina de Estados Finita** (FSM en inglés).

🗣️ Lo que dices en la exposición:

"Los robots tienen una máquina de estados con 7 estados. Empiezan patrullando, cuando detectan al jugador pasan a perseguirlo, y cuando están cerca atacan. Si reciben daño se detienen un momento antes de seguir."

¿Cómo sabe el robot si puede ver a Kai?

Usa algo llamado **Line-of-Sight** (línea de visión). Es como lanzar un rayo invisible desde el robot hasta Kai.

- Si el rayo **no choca con ninguna pared** → el robot puede ver a Kai → ataca
- Si el rayo **choca con una pared** → el robot no puede ver a Kai → no dispara

Esto hace que el jugador pueda **escondarse detrás de paredes** para evitar los disparos.

🗣️ Lo que dices:

"El robot verifica si tiene línea de visión directa al jugador usando un raycast. Si hay una pared entre ellos, no dispara. Esto obliga al jugador a usar la geometría del nivel como cobertura."

¿Qué pasa cuando matas un robot?

- Suma **500 puntos** al marcador
- La cámara tiembla un poco (efecto dramático)
- El robot queda registrado como "derrotado" — si vuelves al nivel, sigue muerto

🗣️ Lo que dices:

"Al destruir un enemigo se suman 500 puntos, la cámara hace un shake, y el sistema de persistencia registra que ese enemigo fue derrotado para que no reaparezca si el jugador vuelve al nivel."

TEMA 2: El sistema de puertas

¿Por qué hay varios tipos de puertas?

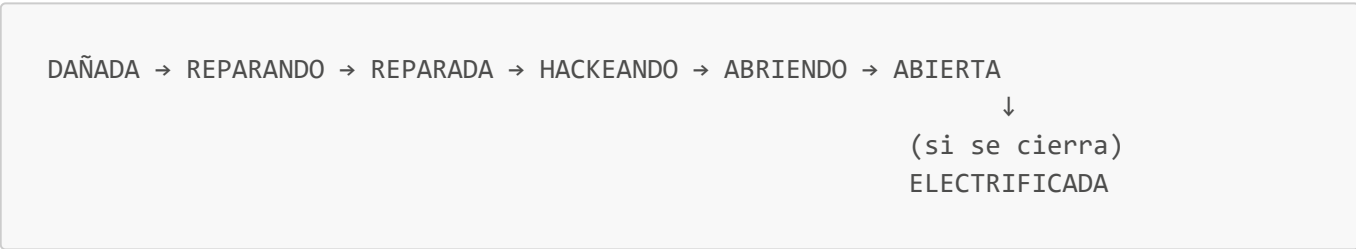
Porque cada tipo de puerta **consume un recurso diferente de Kai**. Esto crea decisiones: si llegás sin energía, no podés hackear la puerta. Si llegás sin stamina, no podés repararla.

Los 6 tipos de puertas

Tipo de puerta	Cómo se abre	Qué consume
Normal	Sola automáticamente	Nada
Dañada	Kai la repara con [F]	Stamina (energía física)
Bloqueada	Kai la hackea con [Q]	Energía (batería)
Electrificada	Primero reparar [F], luego hackear [Q]	Stamina + Energía
De nivel	Presionar [E] para pasar al siguiente nivel	Nada
Con tarjeta	Necesitas encontrar una tarjeta primero	Tarjeta de acceso

La puerta electrificada (la más interesante)

Esta puerta tiene **7 estados internos**, como los robots:



Si Kai la toca cuando está electrificada → recibe daño.

🗣️ Lo que dices:

"El sistema de puertas tiene 6 tipos. Cada uno consume un recurso diferente de Kai. La puerta electrificada es la más compleja — tiene 7 estados internos y requiere primero repararla con la herramienta y luego hackearla. Esto conecta el diseño del nivel con la gestión de recursos del jugador."

El sistema de guardado (bonus — si te queda tiempo)

El juego guarda automáticamente en **3 slots** (3 partidas diferentes). Cada guardado recuerda:

- En qué nivel estás
- Cuántas muertes tuviste
- Cuánto tiempo llevas jugando
- Qué enemigos ya mataste (no reaparecen)
- Qué puertas ya abriste (siguen abiertas)

🗣️ Lo que dices:

"El sistema de guardado tiene 3 slots independientes. Guarda el nivel, las muertes, el tiempo y el estado de cada nivel — si un enemigo fue derrotado o una puerta fue abierta, eso persiste entre

sesiones."

Resumen de lo que dices (orden exacto)

1. Presentarte y hacer la transición:

"Gracias Tatiana. Ahora les voy a explicar cómo funcionan los enemigos y las puertas del juego."

2. Explicar la IA (1.5 min):

"Los robots tienen una máquina de estados con 7 estados: quieto, patrullando, alerta, persiguiendo, atacando, golpeado y destruido. Cambian de estado automáticamente según la situación. Para saber si pueden atacar, usan un raycast — si hay una pared entre el robot y el jugador, no dispara. Al morir suman 500 puntos y quedan registrados como derrotados."

3. Explicar las puertas (1 min):

"El sistema de puertas tiene 6 tipos. Cada uno consume un recurso diferente. La más compleja es la electrificada — tiene 7 estados y requiere reparar y luego hackear. Esto hace que el jugador tenga que gestionar sus recursos antes de enfrentar cada puerta."

4. Explicar el guardado (30 seg):

"El juego guarda en 3 slots. Recuerda el nivel, las muertes, el tiempo y qué enemigos y puertas ya fueron interactuados."

5. Pasar la palabra:

"Con eso les paso a Hector que les va a mostrar el juego funcionando."

Preguntas que te pueden hacer

¿Qué es una máquina de estados?

Es una forma de programar comportamiento donde el objeto solo puede estar en un estado a la vez y cambia de estado según reglas definidas. Es un patrón de diseño clásico en videojuegos.

¿Por qué usar raycast para la visión?

Porque es eficiente. En vez de calcular si hay obstáculos con geometría compleja, lanzamos una línea recta y preguntamos si choca con algo. Es $O(1)$ por frame.

¿Qué pasa si el jugador no tiene energía para hackear?

La puerta muestra un mensaje de "Energía insuficiente" y no hace nada. El jugador tiene que buscar un tubo de energía en el nivel para recargar.

